

AWS EC2에 docker-compose.yml 파일을 이용한 컨테이너 배포실습

실습 > ex06_AWS_docker_compose_useBoard폴더

AWS EC2 ubuntu 생성

인스턴스유형 - t2.small 선택

Amazon Machine Image(AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-0d5bb3742db8fc264 (64비트(Arm))
가상화: hvm ENA 활성화: true 부트 디바이스 유형: ebs

설명
Ubuntu Server 24.04 LTS (HVM), EBS General Purpose (SSD) Volume Type. Support available from Canonical (http://www.ubuntu.com/cloud/services).

Canonical, Ubuntu, 24.04, amd64 noble image

아키텍처: 64비트(x86) AMI ID: ami-0d5bb3742db8fc264 계시 날짜: 2025-03-05 사용자 이름: ubuntu 확인된 금액 검색

인스턴스 유형: t2.small 선택(프리티어아님)

인스턴스 유형: t2.small
제일자리: t2 1 vCPU 2 GiB 메모리 현재 세대: true 온디맨드 Linux 기본 요금: 0.0288 USD 시간당
온디맨드 Ubuntu Pro 기본 요금: 0.0306 USD 시간당 온디맨드 RHEL 기본 요금: 0.0432 USD 시간당
온디맨드 Windows 기본 요금: 0.038 USD 시간당 온디맨드 SUSE 기본 요금: 0.0588 USD 시간당

모든 세대
인스턴스 유형 비교

소프트웨어가 사전 설치된 AMI에는 추가 비용이 적용됩니다.

요약
인스턴스 개수: 1
소프트웨어 이미지(AMI)
Canonical, Ubuntu, 24.04, amd64... 더 보기
ami-0d5bb3742db8fc264
가상 서버 유형(인스턴스 유형)
t2.small
방화벽(보안 그룹)
새 보안 그룹
스토리지(볼륨)
1개의 볼륨 - 8GiB

프리 티어: AWS 계정을 개설한 첫해에 프리 티어 AMI, 매월 750시간의 퍼블릭 IPv4 주소 사용량, 30GiB의 EBS 스토리지, 2백만 I/O, 1GB의 스냅샷 및 100GB의 인터넷 대역폭과 함께 사용하면 매월 750시간의 t2.micro 인스턴스 사용량(또는 t2.micro를 사용할 수 없는 경우 t3.micro)을 제공받습니다.

취소 인스턴스 시작 미리 보기 코드

인스턴스 시작

키 페어(로그인) 정보
키 페어를 사용하여 인스턴스에 안전하게 접속할 수 있습니다. 인스턴스를 시작하기 전에 선택한 키 페어에 대한 액세스 권한이 있는지 확인하세요.

키 페어 이름 - 필수
test-key 새 키 페어 생성

네트워크 설정
네트워크: vpc-029bf33d1054705
서브넷: 기본 설정 없음(가용 영역의 기본 서브넷)
퍼블릭 IP 자동 할당: 활성화
프리 티어 적용 범위를 벗어나는 경우 추가 요금이 적용됩니다.
방화벽(보안 그룹): 보안 그룹 생성
다음 규칙을 사용하여 'launch-wizard-2'이라는 새 보안 그룹을 생성합니다.
☒ 다음에서 SSH 트래픽 허용 인스턴스 연결에 도움 → 22번 원격접속
☐ 인터넷에서 HTTPS 트래픽 허용
예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면
☒ 인터넷에서 HTTP 트래픽 허용
예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면 → tomcat 80 으로 접속예정
소스가 0.0.0.0/0인 규칙은 모든 IP 주소에서 인스턴스에 액세스하도록 허용합니다. 알려진 IP 주소의 액세스만 허용하도록 보안 그룹을 설정하는 것이 좋습니다.

스토리지 구성
1x 30 GiB gp3 부트 볼륨, 3000IOPS, 암호화되지 않음

요약
인스턴스 개수: 1
소프트웨어 이미지(AMI)
Canonical, Ubuntu, 24.04, amd64... 더 보기
ami-0d5bb3742db8fc264
가상 서버 유형(인스턴스 유형)
t2.small
방화벽(보안 그룹)
새 보안 그룹
스토리지(볼륨)
1개의 볼륨 - 30GiB

프리 티어: AWS 계정을 개설한 첫해에 프리 티어 AMI, 매월 750시간의 퍼블릭 IPv4 주소 사용량, 30GiB의 EBS 스토리지, 2백만 I/O, 1GB의 스냅샷 및 100GB의 인터넷 대역폭과 함께 사용하면 매월 750시간의 t2.micro 인스턴스 사용량(또는 t2.micro를 사용할 수 없는 경우 t3.micro)을 제공받습니다.

취소 인스턴스 시작 미리 보기 코드

인스턴스에서 확인

인스턴스 (1) 정보

인스턴스를 속성 또는 (case-sensitive) 태그로 찾기

모든 상태

인스턴스 ID: i-0e9109200ad9aeb4b | 인스턴스 상태: 실행 중 | 인스턴스 유형: t2.small | 상태 검사: 2/2개 검사 통과 | 정보 보기

가용 영역: ap-northeast-2a | 퍼블릭 IPv4 DNS: ec2-43-202-2-110.ap-n... | 퍼블릭 IPv4: 43.202.2.110 | 퍼블릭 IPv6: -

#MobaXterm으로 EC2 원격 접속하기

인스턴스를 속성 또는 (case-sensitive) 태그로 찾기

모든 상태

인스턴스 ID: i-0e9109200ad9aeb4b | 인스턴스 상태: 실행 중 | 인스턴스 유형: t2.small | 상태 검사: 2/2개 검사 통과 | 정보 보기

가용 영역: ap-northeast-2a | 퍼블릭 IPv4 DNS: ec2-43-202-2-110.ap-n... | 퍼블릭 IPv4: 43.202.2.110 | 퍼블릭 IPv6: -

SSH

Basic SSH settings

Remote host: 43.202.2.110 | Specify username: ubuntu | Port: 22

Advanced SSH settings

X11-Forwarding: ☒ | Compression: ☒ | Remote environment: Interactive shell

Execute command: | Do not exit after command ends: ☐

SSH-browser type: SFTP protocol | Follow SSH path (experimental): ☐

☒ Use private key: D:\2025년_privateStudy\20050328 | Expert SSH settings

Execute macro at session start: <none>

생성한 키페어 연결

OK

Cancel

AWS EC2 Ubuntu에 Docker를 설치하기 위해 아래의 명령어를 순서대로 진행한다.

#최신 패키지를 설치하기 위해 패키지 목록을 업데이트 - 최신버전 정보 받아오기

#명령어

```
sudo apt update
```

#Docker를 설치하려면 몇 가지 필수 패키지를 먼저 설치해야 한다.

: Docker 설치에 의존되는 도구들을 미리 준비

: Docker는 HTTPS 저장소를 사용하므로 아래 도구들이 없으면 Docker 설치가 불가능

패키지 이름	역할
apt-transport-https	APT가 HTTPS를 통해 패키지를 다운로드할 수 있게 함
ca-certificates	HTTPS 인증서를 검증하는 데 필요
curl	웹에서 파일을 다운로드할 수 있는 도구
software-properties-common	add-apt-repository 명령어를 사용할 수 있게 해줌

#명령어

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common
```

#Docker의 공식 GPG 키 등록

: Docker에서 제공하는 패키지가 **정상적인지(위조되지 않았는지)** 검증하기 위한 **디지털 서명 키 (GPG)** 를 등록한다. 보안을 위해 APT는 **서명된 패키지만 설치하도록** 되어 있어서, GPG 키를 미리 등록해 놓아야 APT가 Docker를 **신뢰하고 설치**한다.

#명령어

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee  
/etc/apt/trusted.gpg.d/docker.asc
```

#Docker의 공식 저장소를 APT 소스에 추가

: APT가 Docker 패키지를 다운로드할 **공식 저장소 URL**을 등록

: Ubuntu 기본 저장소에는 Docker의 **최신 버전이 없기 때문에** Docker 공식 저장소를 등록해야 최신 버전의 docker-ce를 설치할 수 있다.

#명령어

```
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu  
$(lsb_release -cs) stable"
```

#새로운 Docker 저장소가 추가되었으므로, 패키지 목록을 다시 업데이트

위 단계에서 **Docker 저장소를 새로 등록**했기 때문에, 다시 apt update로 **해당 저장소의 패키지 정보도 불러와야** 한다. docker-ce가 이제 APT에서 검색 가능해졌으므로, 목록을 다시 갱신해 줘야 설치 가능하다.

#명령어

```
sudo apt update
```

#Docker를 설치

: docker-ce는 Docker의 Community Edition이다.

: Docker 데몬, CLI 등 핵심 도구들을 함께 설치한다.

#명령어

```
sudo apt install docker-ce
```

#Docker가 자동으로 시작되지만, 확인을 위해 서비스를 시작할 수 있다.

#명령어

```
sudo systemctl start docker
```

#Docker 서비스가 정상적으로 실행되고 있는지 확인

#명령어

```
sudo systemctl status docker
```

#Docker 설치가 완료되었는지 확인하려면 버전을 확인

#명령어

```
docker --version
```

```
ubuntu@ip-172-31-12-26:~$ docker --version  
Docker version 28.1.1, build 4eba377
```

#현재 사용자로 root 권한을 부여 받은 후, root 사용자로 전환한다

#명령어

```
sudo su
```

```
ubuntu@ip-172-31-12-26:~$ sudo su  
root@ip-172-31-12-26:/home/ubuntu#
```

폴더생성

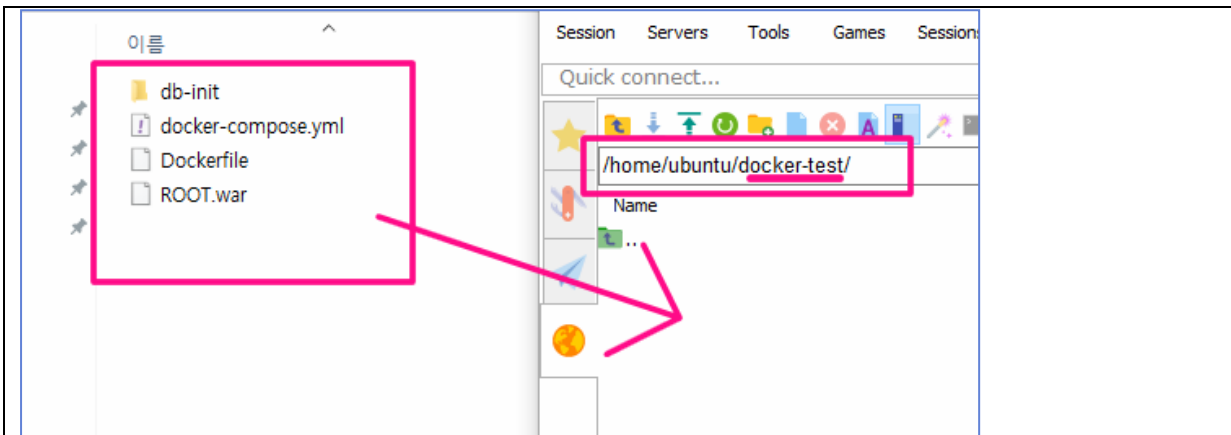
#명령어

```
mkdir docker-test
```

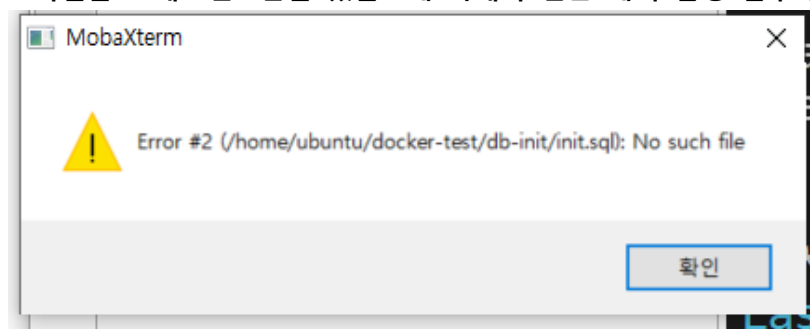
```
mkdir upload
```

```
root@ip-172-31-12-26:/home/ubuntu# mkdir docker-test  
root@ip-172-31-12-26:/home/ubuntu# mkdir upload  
root@ip-172-31-12-26:/home/ubuntu# ls  
docker-test  upload
```

mobaxterm 에서 SFTP를 이용하여 드래그 앤 드롭으로 로컬pc의 파일을
aws의 docker-test폴더에 복사한다.



파일을 드래그앤드롭을 했을 때 아래와 같은 에러 발생 할수 있다.



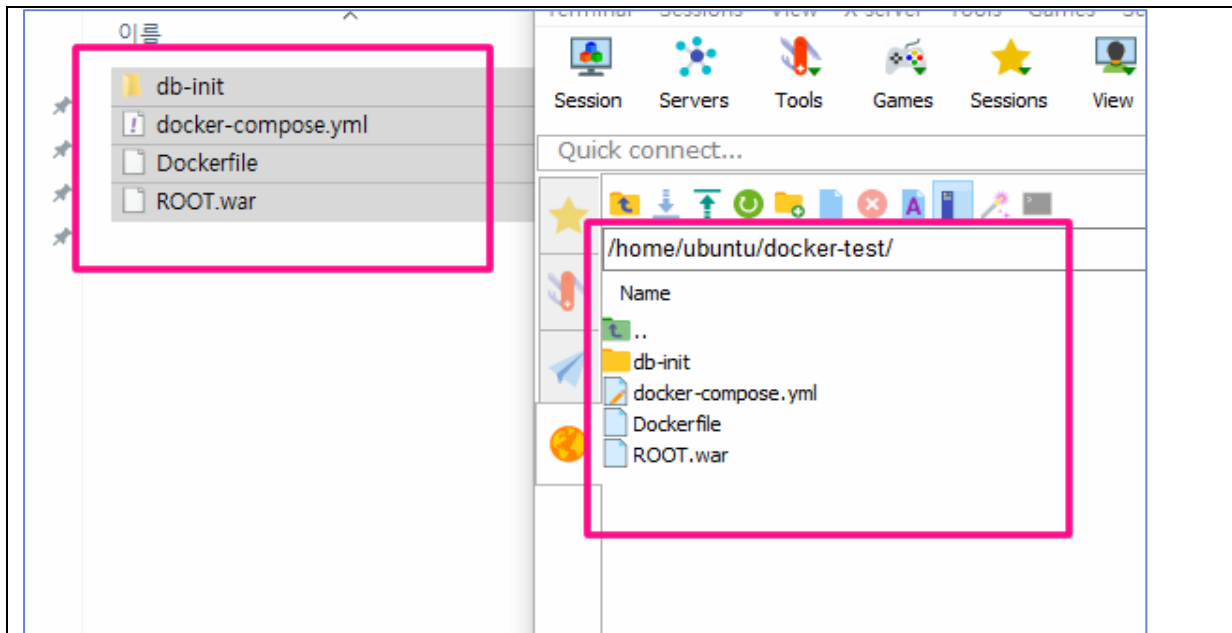
sudo su 명령어 이후 root권한으로 docker-test와 upload 디렉토리를 생성했기 때문에 일반 사용자 ubuntu는 이 폴더에 파일을 생성하거나 수정할 권한이 없다.

그래서 소유권을 ubuntu에게 넘겨야, ubuntu 계정으로 로그인한 사용자가 해당 디렉토리에서 자유롭게 작업 가능해진다.

#명령어

```
chown ubuntu:ubuntu /home/ubuntu/docker-test
```

드그래인드랍 다시 해본다.



폴더 docker-test로 이동

#명령어

```
cd docker-test
```

Docker Compose V2부터는 docker-compose 대신 docker compose

#명령어

```
docker compose version
```

ls 명령어로 현재 폴더에 docker-compose.yml 있는지 확인해본다.

```
root@ip-172-31-12-26:/home/ubuntu/docker-test# ls
Dockerfile  ROOT.war  db-init  docker-compose.yml
```

#docker-compose실행하기

#명령어

```
docker compose up --build -d
```

```
[+] Running 5/5
✓ tomcat Built 0.0s
✓ Network docker-test_web_net Created 0.1s
✓ Volume "docker-test_mysql_vol" Created 0.0s
✓ Container my_sql Started 0.4s
✓ Container my_tomcat Started 0.7s
```

실행중인 프로세스 확인하기

#명령어

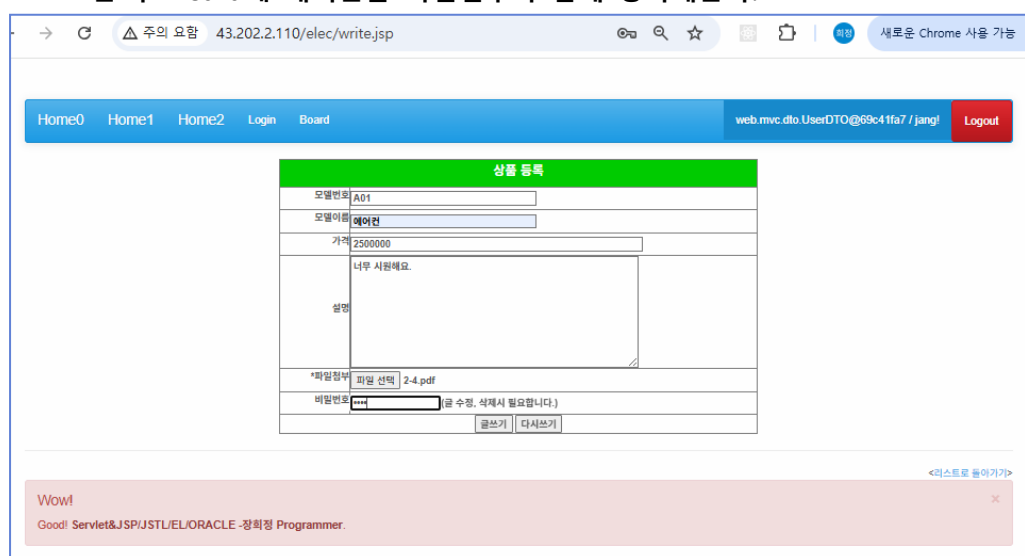
```
docker ps -a
```

```
root@ip-172-31-12-26:/home/ubuntu/docker-test# docker ps -a
CONTAINER ID   IMAGE          NAMES      COMMAND                  CREATED        STATUS        PORTS
db192bfe2f5d   docker-test-tomcat   my_tomcat   "catalina.sh run"       About a minute ago   Up About a minute   0.0.0.0:80->8080/tcp, [::]:80->8080/tcp
18415964540b   mysql:8.4         my_sql     "docker-entrypoint.s..." About a minute ago   Up About a minute   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
root@ip-172-31-12-26:/home/ubuntu/docker-test#
```

브라우저를 열고 ec2 public ip로 접속해본다.



로그인 후 Board에 게시물을 파일첨부와 함께 등록해본다.



volume으로 연결한 파일업로드 폴더에 첨부된 파일이 저장 된 것을 확인한다.

The screenshot shows a web browser window on the left and a terminal window on the right. The web browser displays a 'Board' page with a '상품(Electronic) LIST' table. The table has columns for '가격' (Price), '날짜' (Date), '첨부파일' (Attachment), and '파일용량' (File Size). The '첨부파일' column shows a file named '2-4.pdf' with a size of '1,303,653 byte'. The terminal window shows the command 'ls' being executed in the '/home/ubuntu/upload/' directory, listing the file '2-4.pdf'.

가격	날짜	첨부파일	파일용량
2,500,000	2025-05-02 10:01:28	2-4.pdf	1,303,653 byte
1,300,000	2025-05-02 09:55:35		0 byte
1,700,000	2025-05-02 09:55:35		0 byte
1,000,000	2025-05-02 09:55:35		0 byte

```

Container my_tomcat Started
root@ip-172-31-12-26:/home/ubuntu/docker-test# d
CONTAINER ID IMAGE COMMAND
PORTS
db192bfe2f5d docker-test-tomcat "catalina.sh
a minute 0.0.0.0:80->8080/tcp, [::]:80->8080/t
18415964540b mysql:8.4 "docker-entr
a minute 0.0.0.0:3306->3306/tcp, [::]:3306->33
root@ip-172-31-12-26:/home/ubuntu/docker-test# d
CONTAINER ID IMAGE COMMAND
db192bfe2f5d docker-test-tomcat "catalina.sh
:80->8080/tcp my_tomcat
18415964540b mysql:8.4 "docker-entr
:]:3306->3306/tcp, 33060/tcp my_sql
root@ip-172-31-12-26:/home/ubuntu/docker-test# c
root@ip-172-31-12-26:/home/ubuntu# cd up
bash: cd: up: No such file or directory
root@ip-172-31-12-26:/home/ubuntu# cd upload/
root@ip-172-31-12-26:/home/ubuntu/upload# ls
2-4.pdf
root@ip-172-31-12-26:/home/ubuntu/upload#
  
```

my_tomcat 컨테이너 내부로 들어가서 파일 확인하기

#명령어

```
docker exec -it my_tomcat /bin/bash
```




#docker-compose 삭제

#명령어 - 볼륨까지 삭제는 -v옵션

```
docker compose down -v
```

#명령어 - 볼륨까지 삭제안함.

```
docker compose down
```

#도메인 연결하기

내도메인.한국 로그인 > 도메인관리 > 도메인 수정

도메인	grace24.o-r.kr	
웹 포워딩 (Redirect)		
☐ 웹 포워딩	.grace24.o-r.kr	http://
단일페이지 (HTML)		
☐ 단일페이지	.grace24.o-r.kr	
<pre><html> <head></head> <body></body> </html></pre>		
고급설정 (DNS)		
☒ IP연결 (A)	.grace24.o-r.kr	43.202.2.110

ec2 public ip

http://grace24.o-r.kr/

← → ↻ △ 주의 요함 grace24.o-r.kr ☆ □ | 새로운 Chrome 사용 가능

[Home0](#)
[Home1](#)
[Home2](#)
[Login](#)
[Board](#)

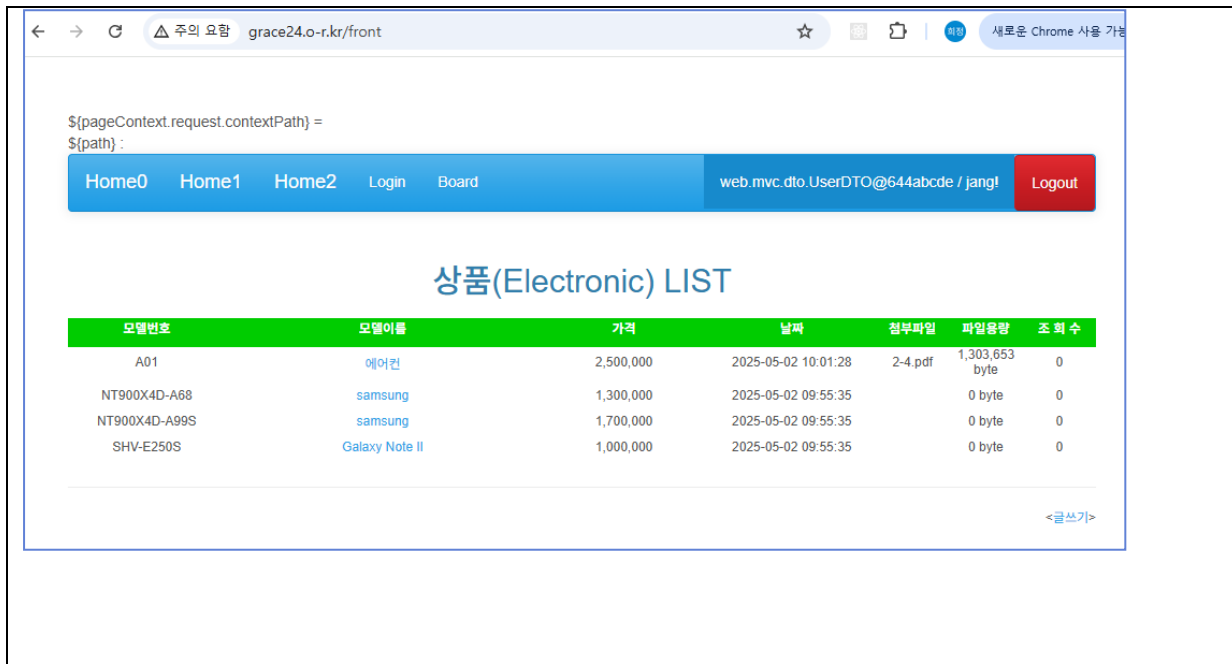
KOSTA My Sevlet & Jsp Final !!!!

Sevlet&Jsp Final Wow!

- 주요기능
 - 회원관리(로그인 / 로그아웃) - MEMBER Table
 - 자료실 형태의 게시판관리 (등록, 수정, 삭제, 검색, 다운로드) - Electronics Table
- 주요 기술 및 환경
 - Refactoring MVC구조
 - Filter - Session유무 체크
: Board에 대한 접근은 인증된 사용자만 가능
 - ServletContextListener - 사전 초기화 작업
 - Servlet Upload + Download
 - 사용자 정의 Exception - AuthenticationException
 - Action Tag include - Layout Template
 - Bootsrap UI
 - MySql 8.x
 - ApacheTomcat 10.1

Wow!

Good! Servlet&JSP/JSTL/EL/ORACLE -장희정 Programmer.



제공된 파일 살펴보기

- Step14_AWS_UserElec 프로젝트 변경된 부분
 - webapp/META-INF/context.xml 설정
 - ElectronicsController.java / DownloadServlet.java 파일 경로 수정
 - DownloadServlet.java 다운로드 경로 수정
- step18_AWS_user_elec 프로젝트 Export하기 - ROOT.war
- db-init폴더
 - init.sql
- Dockerfile 파일
- docker-compose.yml 파일

Step14_AWS_UserElec 프로젝트 변경된 부분

context.xml 설정 변경

개발모드에서는 db를 local로 사용 했지만 ec2배포에서는 mysql container로 연결해야한다.

webapp > META-INF > context.xml

```
<Resource name="jdbc/mysql"
  auth="Container"
  type="javax.sql.DataSource"
  driverClassName="com.mysql.cj.jdbc.Driver"
  url="jdbc:mysql://my_sql:3306/web_basic"
  username="root"
  password="admin"
  maxTotal="20"
  maxIdle="10"
  maxWaitMillis="-1"/>
```

mysql의 컨테이너이름

파일업로드된 자료가 container가 삭제되어도 남아 있어야 하기에 volume 사용해야 한다.

docker-compose.yml파일에

```
volumes:
  - /home/ubuntu/upload:/usr/local/tomcat/webapps/save # ← 업로드된 파일을 호스트에 저장
```

Ec2의 ubuntu 홈디렉토리에 upload를 폴더를 만들고 container의 /webapps/save와 마운트한다.

주의사항 : /webapps/ROOT/save 로 설정하면 volume마운트를 위해서 Dockerfile에

```
mkdir -p /usr/local/tomcat/webapps/ROOT/save
```

설정한다.

이로 인해 , ROOT에 안에 save폴더가 만들어지면서 ROOT가 생성되어진다.

Tomcat은 ROOT.war 를 자동 압축 해제 할 때 ROOT폴더가 이미 존재 하면 자동으로 압축 해제하지 않는다. 때문에 volume을 ROOT폴더 안에 있는 폴더와 마운트하지 않도록 한다.

자바소스 업로드 경로 수정

ElectronicsControlller.java – insert 메소드부분

```

86
87 String rootPath = request.getServletContext().getRealPath("/"); // webapps/ROOT
88 File rootDir = new File(rootPath);
89 File webappsDir = rootDir.getParentFile(); // webapps
90 File saveDir = new File(webappsDir, "save"); //webapps/save
91
92
93 //String saveDir = "C:\\Edu\\save";
94 System.out.println("saveDir = " + saveDir);
95 System.out.println("실제 저장 경로: " + saveDir.getAbsolutePath());
96
97 if (fileName!=null && !fileName.equals("")) {
98     part.write(new File(saveDir, fileName).getAbsolutePath()); //서버폴더에 파일 저장=업로드
99
100     elec.setFname(fileName);
101     elec.setFsize( (int)part.getSize() );
102 }
103

```

ElectronicsControlller.java – delete 메소드부분

```

199
200 public ModelAndView delete(HttpServletRequest request, HttpServletResponse response)
201     throws Exception {
202     //전송되는 2개받기
203     String modelNum = request.getParameter("modelNum");
204     String password = request.getParameter("password");
205
206     String rootPath = request.getServletContext().getRealPath("/"); // webapps/ROOT
207     File rootDir = new File(rootPath);
208     File webappsDir = rootDir.getParentFile(); // webapps
209     File saveDir = new File(webappsDir, "save"); //webapps/save
210
211     //첨부된 파일을 삭제하는 경우라면 삭제가 되었을때 폴더에서 파일도 삭제한다. - 이 기능을 service한다.
212     //String saveDir = request.getServletContext().getRealPath("/save");
213
214     elecService.delete(modelNum, password, saveDir.getAbsolutePath());
215
216     return new ModelAndView("front", true);
217 }

```

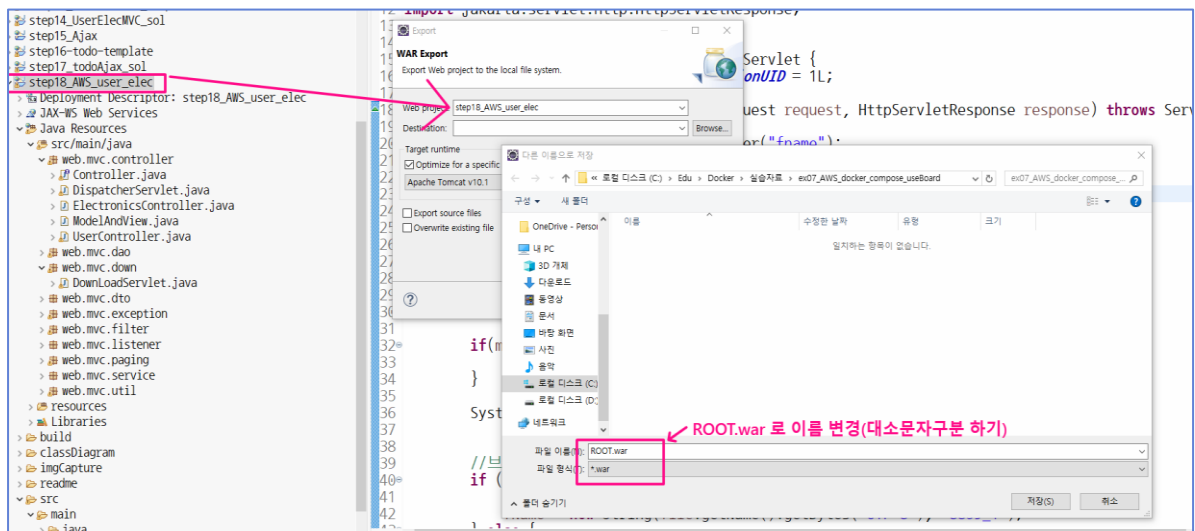
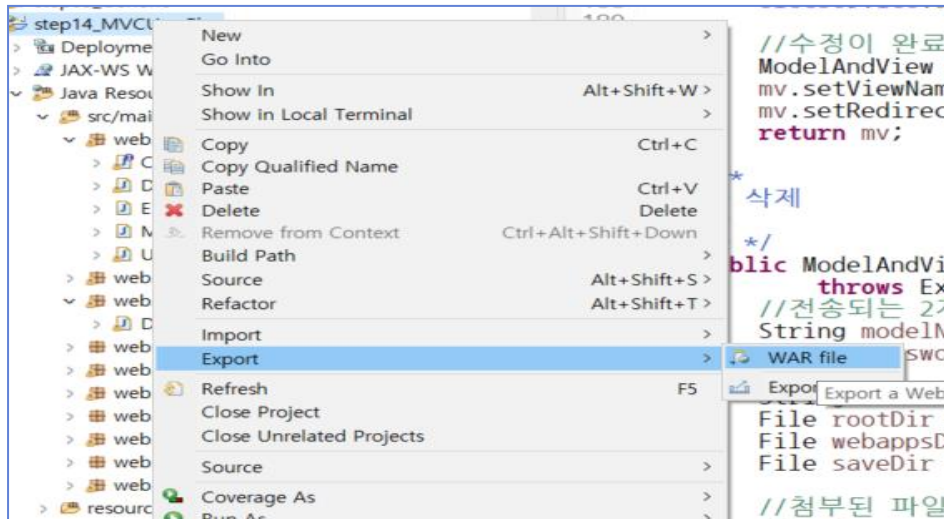
DownloadServlet.java – service 메소드부분

```

14 @WebServlet("/download")
15 public class DownloadServlet extends HttpServlet {
16     private static final long serialVersionUID = 1L;
17
18     protected void service(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
19         //1. 넘어오는 파일의 이름을 받기
20         String fName = request.getParameter("fname");
21
22         //2. 저장폴더의 실제 경로를 얻어오기
23
24         String rootPath = request.getServletContext().getRealPath("/"); //webapps/ROOT
25         File rootDir = new File(rootPath);
26         File webappsDir = rootDir.getParentFile(); //webapps
27         File saveDir = new File(webappsDir, "save"); //webapps/save
28
29
30         File file = new File(saveDir, fName);
31

```

Step14_AWS_UserElec프로젝트 Export하기 – ROOT.war



db-init폴더 > init.sql

```

1 • use web_basic;
2
3 • CREATE TABLE IF NOT EXISTS Electronics(
4     model_num varchar(15) primary key,
5     model_name varchar(20) not null,
6     price int,
7     description varchar(100),
8     password varchar(20) not null,
9     writeday datetime not null,
10    readnum int,
11    fname varchar(50),
12    fsize int
13 );
14
15 • insert into Electronics values('NT900X4D-A68','samsung',1300000,'Windows 8','1111',now(),0,null,0);
16 • insert into Electronics values('SHV-E250S','Galaxy Note II',1000000,'Wi-Fi bluetooth 4.0','1111',now(),0,null,0);
17 • insert into Electronics values('NT900X4D-A99S','samsung',1700000,'Windows 8','1111',now(),0,null,0);
18
19 • CREATE TABLE IF NOT EXISTS replies(
20     reply_num int primary key auto_increment,
21     reply_content varchar(100) not null,
22     reply_regdate datetime,
23     parent_model_num varchar(15) ,
24     foreign key(parent_model_num) references Electronics(model_num)
25 );
26
27 • insert into replies(reply_content,reply_regdate , parent_model_num) values( 'NT900X4D-A68 first reply', now() , 'NT900X4D-A68');
28 • insert into replies(reply_content,reply_regdate , parent_model_num) values( 'NT900X4D-A68 second reply', now() , 'NT900X4D-A68');
29 • insert into replies(reply_content,reply_regdate , parent_model_num) values( 'NT900X4D-A68 third reply', now() , 'NT900X4D-A68');
30
31 • insert into replies(reply_content,reply_regdate , parent_model_num) values( 'NT900X4D-A99S first reply', now() , 'NT900X4D-A99S');
32 • insert into replies(reply_content,reply_regdate , parent_model_num) values( 'NT900X4D-A99S second reply', now() , 'NT900X4D-A99S');
33
34 • CREATE TABLE IF NOT EXISTS users(
35     user_id varchar(10) primary key,
36     pwd varchar(10),
37     name varchar(10)
38 );
39
40 • insert into users values('jang', '1234', 'heejung');
41 • insert into users values('lee', '1234', 'Lee GaHyun');

```

Dockerfile 파일

```

1 FROM tomcat:10
2
3 # save 디렉토리 미리 생성
4 RUN mkdir -p /usr/local/tomcat/webapps/save
5
6 # WAR 복사
7 COPY ROOT.war /usr/local/tomcat/webapps/
8

```

FROM tomcat:10

: 기반 이미지: Apache Tomcat 10을 사용해서 컨테이너를 만들겠다는 의미

: Tomcat은 /usr/local/tomcat/webapps/ 경로에 .war 파일을 배포하면 자동으로 압축 해제 함

RUN mkdir -p /usr/local/tomcat/webapps/save

: Tomcat 배포 경로에 save 폴더를 미리 생성

: 업로드 폴더(/save)를 만들기 위해서 사용합니다. 나중에 이 경로를 호스트와 마운트할 수 있도록 준비함.

COPY ROOT.war /usr/local/tomcat/webapps/

: WAR 파일을 컨테이너 내부로 복사

: Tomcat은 이 파일을 자동으로 /webapps/ROOT/로 압축 해제함

Docker-compose.yml 파일

```

1  version: '3.8'
2
3  services:
4    mysql:
5      image: mysql:8.4
6      container_name: my_sql
7      environment:
8        MYSQL_ROOT_PASSWORD: "admin"
9        MYSQL_DATABASE: "web_basic"
10       TZ: "Asia/Seoul"
11     volumes:
12       - mysql_vol:/var/lib/mysql # 데이터 영속화: mysql_vol → MySQL의 실제 DB 데이터가 여기에 저장됨
13       - ./db-init:/docker-entrypoint-initdb.d # 초기 SQL 실행: ./db-init/init.sql 파일 등을 통해 컨테이너 초기 구동 시 테이블 생성 가능(SQL 스크립트 자동 실행 경로)
14       - /etc/localtime:/etc/localtime:ro # 시간 동기화: 호스트의 시간을 컨테이너와 동일하게 맞춤
15     ports:
16       - "3306:3306"
17     networks:
18       - web_net # tomcat과 같은 네트워크에 연결되어 같은 내부 이름으로 통신할 수 있다 (예: my_sql로 접근)
19
20    tomcat:
21      build: . # 현재 디렉토리에 있는 Dockerfile을 기반으로 이미지를 새로 빌드
22      container_name: my_tomcat
23      ports:
24        - "80:8080"
25      networks:
26        - web_net
27      depends_on: # mysql 컨테이너가 먼저 시작된 후 tomcat이 시작된다.
28        - mysql
29      volumes:
30        - /home/ubuntu/upload:/usr/local/tomcat/webapps/save # Tomcat 컨테이너의 /webapps/save 경로와 호스트의 /home/ubuntu/upload 경로를 연결
31
32    volumes:
33      mysql_vol:
34
35    networks:
36      web_net: # Tomcat과 MySQL이 같은 네트워크로 묶여 내부 DNS 이름으로 서로 접근 가능 (my_sql:3306 등)
37

```

Dockerfile ↔ docker-compose 실행 순서

전체 실행 순서

1. docker-compose up --build -d

2. build : . 옵션을 발견 → 현재 디렉토리에서 Dockerfile 을 실행해서 이미지 생성
 - 이때 Dockerfile 의 FROM, RUN, COPY 등이 실행됨
3. 이미지가 완성되면 my_tomcat, my_sql 컨테이너가 생성됨
4. mysql 이 먼저 실행되고, 이후 depends_on 된 tomcat 이 실행됨
5. tomcat 은 내부적으로 /webapps/ROOT 에 WAR 를 배포하고, /webapps/save 는 호스트와 연결됨

참고하기

Docker CLI로 작성할 수 있는 명령어는 전부 **docker-compose.yml** 파일로 옮길 수 있다. 반대로 **docker-compose.yml**에 작성한 모든 값은 **Docker CLI**로 나타낼 수 있다. 이를 편하게 변환해주는 사이트가 존재한다.

Docker CLI → compose.yml로 변환

: <https://www.composerize.com/>

compose.yml → Docker CLI로 변환

: <https://www.decomposerize.com/>

AWS EC2 Linux Docker JSTL 이슈

..ClassNotFoundException: org.apache.jsp.common.header_jsp가 난다.

로컬 PC Docker에서 되고 AWS EC2 Docker환경에서 안되는 이유

같은 Dockerfile에 FROM tomcat:10인데도 EC2와 로컬에서 다르게 동작할 수 있다.

이건 Docker 태그가 '움직이는'(mutable) 탓이다. tomcat:10은 고정 버전이 아니라 10 계열의 최신을 가리키므로, 시점·아키텍처·캐시 상태에 따라 **전혀 다른 이미지를 받을 수 있다.**

여기에 **캐시/아키텍처/플 정책**까지 겹치면 로컬=정상, EC2=오류가 쉽게 생긴다.

해결책

1. **lib폴더에** jakarta.servlet.jsp.jstl-api-3.0.0.jar , jakarta.servlet.jsp.jstl-3.0.1.jar **추가** 하고
jstl.jar, standard.jar, jakarta.servlet.jsp.jstl-2.0.0.jar, jakarta.servlet.jsp.jstl-api-2.0.0.jar **삭제**
2. jsp문서에서 JSTL선언문 **uri 변경**

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
```

```
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
```

라이브러리 브라우저로 직접 받기 (클릭)

3. API 3.0.0
<https://repo1.maven.org/maven2/jakarta/servlet/jsp/jstl/jakarta.servlet.jsp.jstl-api/3.0.0/jakarta.servlet.jsp.jstl-api-3.0.0.jar>
4. 구현 3.0.1 (GlassFish)
<https://repo1.maven.org/maven2/org/glassfish/web/jakarta.servlet.jsp.jstl/3.0.1/jakarta.servlet.jsp.jstl-3.0.1.jar>